Order & Product Inventory Management System

Complete Technical Documentation by Nathanael L. Larida

Table of Contents

1. System Overview

BiteBox is a desktop Point-of-Sale (POS) and inventory management application built for small to mid-sized retail and food businesses. It replaces manual, paper-based order-taking and inventory tracking with a single, integrated desktop application.

A single **Sign In** window serves as the entry point. The system automatically routes the user to the correct portal based on their credentials:

- **Staff credentials** → Staff/Admin Management Portal

- **Customer credentials** → Customer Self-Service Portal

Closing either portal returns the user to the Sign In window so the next person can log in without restarting the application.

2. Tech Stack

Component	Library / Tool	Purpose
Language	**Python 3.11+**	Core language; type hints used throughout
GUI Framework	**PyQt6**	Desktop GUI windows, widgets, dialogs
Charts	**PyQt6-Charts**	Bar, pie, and line charts on the dashboard
ORM	**SQLAlchemy 2.0**	Database models and queries
Database	**SQLite**	Local file-based persistence (inventory.db)
Password Security	**bcrypt**	Secure password hashing (never stored in plain text)
PDF Generation	**reportlab**	PDF report exports
Vector Icons	**qtawesome**	Icons in the customer portal header
Image Support	**Pillow**	Decodes product images reliably
Email	**smtplib / email**	HTML order-confirmation emails via SMTP

3. Project Structure

```
BiteBox_OrderAndProductInventoryManagementSystem/
│
├── main.py                    # Application entry point; login loop
├── config.py                  # Path constants and config loader/saver
├── config.json                # Runtime settings (store info, SMTP, theme)
├── requirements.txt           # Python dependency list
│
├── database/
│   ├── db_manager.py          # Engine, session factory, init_db(), migrations
│   ├── models.py              # SQLAlchemy ORM table definitions
│   └── seed_data.py           # First-run seed: default accounts, products,
promos
│
├── models/                    # Pure Python dataclasses (data transfer objects)
│   ├── customer.py
│   ├── order.py
│   ├── product.py
│   ├── promotion.py
│   └── staff.py
│
├── services/                  # Business logic layer (no GUI code here)
```

```
│   ├── auth_service.py       # Staff login, password hashing
│   ├── cart_service.py       # In-memory Cart class
│   ├── category_service.py   # Category CRUD
│   ├── customer_auth_service.py  # Customer registration and login
│   ├── customer_service.py   # Customer queries and profile updates
│   ├── email_service.py      # HTML email builder and SMTP sender
│   ├── exporters.py          # Abstract BaseExporter + concrete subclasses
│   ├── order_service.py      # Order placement, status transitions, audit log
│   ├── product_service.py    # Product CRUD, image management
│   ├── promotion_service.py  # Promotion CRUD and code validation
│   ├── report_service.py     # Products/orders report queries, CSV/PDF helpers
│   └── staff_service.py      # Staff CRUD
│
├── gui/                      # PyQt6 GUI — Staff/Admin portal
│   ├── main_window.py        # Staff portal window with sidebar navigation
│   ├── dashboard_tab.py      # KPI cards + bar/pie/line charts
│   ├── products_tab.py       # Product list, add/edit/delete/restore
│   ├── categories_tab.py     # Category management
│   ├── orders_tab.py         # Order list, status updates, detail view
│   ├── new_order_window.py   # Staff new-order flow (product grid + cart)
│   ├── customers_tab.py      # Customer list and profile view
│   ├── promotions_tab.py     # Promotion CRUD
│   ├── reports_tab.py        # Products and orders reports with export
│   ├── staff_tab.py          # Staff CRUD (Admin only)
│   ├── settings_tab.py       # Store info, SMTP config, theme toggle
│   │
│   ├── customer/             # Customer self-service portal
│   │   ├── customer_login_window.py   # Shared login/register dialog
│   │   ├── customer_main_window.py    # Customer portal shell (sidebar +
header)
│   │   ├── menu_tab.py                # Hero banner, category filter, product
grid
│   │   ├── cart_panel.py              # Cart + checkout 2-step dialog
│   │   ├── orders_tab.py              # Customer order history
│   │   ├── profile_tab.py             # Edit profile, delivery address,
password
│   │   ├── notifications_panel.py     # Order-status activity feed
│   │   ├── branding_panel.py          # Store branding display
│   │   ├── flow_layout.py             # Wrapping grid layout for product cards
│   │   └── assets_loader.py           # Image/icon helpers for customer portal
│   │
│   └── widgets/              # Reusable GUI components
│       ├── data_table.py      # Sortable/searchable table widget
│       ├── stat_card.py       # KPI metric card
│       ├── product_card.py    # Product thumbnail card for the grid
│       ├── badged_icon_button.py  # Icon button with numeric badge overlay
│       ├── toast.py           # Fade-in/out toast notification
│       └── validators.py      # Form-field validators + error marking
│
└── assets/
    ├── styles/
    │   ├── theme.qss          # Light theme stylesheet
    │   └── theme_dark.qss     # Dark theme stylesheet
    ├── icons/                 # Custom icon files
    └── images/
        └── customer/          # Hero banners, product images
```

4. System Architecture

The application follows a **three-layer architecture**:

```
+-------------------------------------+
|           GUI Layer (gui/)          |  <- PyQt6 windows and widgets
|  Staff Portal     Customer Portal   |     Only handles display and user input
+-----------------+-------------------+
                  | calls
+-----------------v-------------------+
|        Service Layer (services/)    |  <- Business logic
|  auth . order . product . cart ...  |     Validates rules, orchestrates data
+-----------------+-------------------+
                  | reads/writes
+-----------------v-------------------+
|       Database Layer (database/)    |  <- SQLAlchemy ORM on SQLite
|   models.py  .  db_manager.py       |     Persistent storage in inventory.db
+-------------------------------------+
```

Data flow example — customer places an order:

1. Customer clicks **Place Order** in the CartPanel (GUI layer).
2. CartPanel calls order_service.place_order(...) (Service layer).
3. order_service validates stock, computes totals, writes the Order and OrderItem rows, logs the initial Pending status history entry, and decrements quantity_on_hand (Database layer).
4. On success, email_service.send_order_confirmation(...) fires an HTML email.
5. The GUI receives the new order_id and shows a toast notification.

5. Database Schema

Entity-Relationship Overview

```
categories --< products >-- product_option_groups --< product_option_items
                 |
                 |
staff --+        |       +-- customers
        +---< orders >--+
                 |
                 +--< order_items >-- order_item_options
                 +--< order_status_history
                 +--< payment_logs

promotions >--< promotion_products
```

Table Definitions

`categories`

Column	Type	Notes
`category_id`	INTEGER PK	Auto-increment
`category_name`	VARCHAR(100)	Unique
`is_deleted`	BOOLEAN	Soft-delete flag
`created_at`	DATETIME	

`products`

Column	Type	Notes
`product_id`	INTEGER PK	Auto-increment
`product_name`	VARCHAR(200)	
`product_description`	TEXT	Optional
`product_price`	FLOAT	Base price
`category_id`	FK → categories	Optional
`quantity_on_hand`	INTEGER	Current stock
`quantity_sold`	INTEGER	Cumulative sold count
`product_image_path`	VARCHAR(500)	Path to copied image file
`is_active`	BOOLEAN	Visible in customer menu when True
`is_deleted`	BOOLEAN	Soft-delete; recoverable by admin
`deleted_by`	VARCHAR(100)	Staff email who deleted it
`deleted_at`	DATETIME	
`created_at`	DATETIME	
`updated_at`	DATETIME	Auto-updates on save

`product_option_groups`

Column	Type	Notes
`group_id`	INTEGER PK	
`product_id`	FK → products	
`group_name`	VARCHAR(100)	e.g. "Size", "Temperature"

`is_required`	BOOLEAN	Customer must pick one if True
`max_choices`	INTEGER	1 = radio buttons, >1 = checkboxes

`product_option_items`

Column	Type	Notes
`item_id`	INTEGER PK	
`group_id`	FK → product_option_groups	
`option_name`	VARCHAR(100)	e.g. "Large", "Hot"
`additional_price`	FLOAT	Added on top of base price

`customers`

Column	Type	Notes
`customer_id`	INTEGER PK	
`first_name`	VARCHAR(100)	
`last_name`	VARCHAR(100)	
`email`	VARCHAR(200)	Unique
`contact_number`	VARCHAR(50)	Optional
`address`	TEXT	Legacy free-form notes
`street`	VARCHAR(200)	Structured delivery address
`city`	VARCHAR(100)	
`province`	VARCHAR(100)	
`postal_code`	VARCHAR(20)	
`landmark`	VARCHAR(200)	
`password_hash`	VARCHAR(255)	Nullable for walk-in customers
`is_active`	BOOLEAN	Deactivated accounts cannot log in
`created_at`	DATETIME	

`staff`

Column	Type	Notes
`staff_id`	INTEGER PK	
`first_name` / `last_name`	VARCHAR(100)	
`email`	VARCHAR(200)	Unique
`password_hash`	VARCHAR(255)	bcrypt
`role`	VARCHAR(20)	"Admin" or "Staff"
`is_active`	BOOLEAN	
`last_login`	DATETIME	Updated on successful login
`created_at`	DATETIME	

`promotions`

Column	Type	Notes
`promotion_id`	INTEGER PK	
`promotion_name`	VARCHAR(200)	
`promotion_description`	TEXT	
`discount_type`	VARCHAR(20)	"Percentage" or "FixedAmount"
`discount_value`	FLOAT	% amount or fixed amount
`minimum_order_amount`	FLOAT	Cart must meet this threshold
`code`	VARCHAR(50)	Unique redemption code
`usage_limit`	INTEGER	NULL = unlimited
`used_count`	INTEGER	Incremented each use
`start_date` / `end_date`	DATE	NULL = no restriction
`is_active`	BOOLEAN	

`orders`

Column	Type	Notes
`order_id`	INTEGER PK	
`customer_id`	FK → customers	NULL for walk-ins

`staff_id`	FK → staff	NULL for customer self-orders
`order_date`	DATETIME	
`subtotal`	FLOAT	Sum of all line items
`discount_amount`	FLOAT	Applied promotion discount
`total_amount`	FLOAT	subtotal − discount
`order_type`	VARCHAR(20)	"Dine-In", "Takeout", "Delivery"
`order_status`	VARCHAR(30)	Status workflow (see below)
`payment_method`	VARCHAR(30)	e.g. Cash, GCash, Card
`payment_status`	VARCHAR(20)	"Pending" or "Paid"
`voucher_code`	VARCHAR(50)	Promo code used
`delivery_notes`	TEXT	Address / instructions
`email_sent_to`	VARCHAR(200)	Recorded after confirmation email
`email_sent_at`	DATETIME	

Order Status Workflow:

```
Pending --> Processing --> ReadyForPickup --> Completed
                    +--> ReadyForDelivery --> Completed
(Cancel is available from any non-terminal status)
```

`order_items`

Column	Type	Notes
`order_item_id`	INTEGER PK	
`order_id`	FK → orders	
`product_id`	FK → products	Nullable (product may be deleted later)
`product_name`	VARCHAR(200)	Snapshot at time of order
`quantity`	INTEGER	
`unit_price`	FLOAT	Snapshot including selected option prices
`subtotal`	FLOAT	unit_price × quantity

`order_item_options`

Column	Type	Notes
`id`	INTEGER PK	

`order_item_id`	FK → order_items	
`option_name`	VARCHAR(100)	e.g. "Large", "Hot"
`additional_price`	FLOAT	

`order_status_history`

Audit trail of every status change.

Column	Type	Notes
`history_id`	INTEGER PK	
`order_id`	FK → orders	
`from_status`	VARCHAR(30)	NULL on initial placement
`to_status`	VARCHAR(30)	
`changed_by`	VARCHAR(100)	Staff name or "Customer"
`notes`	TEXT	Optional comment
`created_at`	DATETIME	

`payment_logs`

Records each payment transaction against an order.

6. User Roles & Access Control

Feature	Admin	Staff	Customer
Dashboard	Yes	Yes	No
Products — View	Yes	Yes	Yes (active only, via Menu)
Products — Add/Edit	Yes	Yes	No
Products — Delete/Restore	Yes	Yes	No
Categories — CRUD	Yes	Yes	No
New Order (walk-in)	Yes	Yes	No
Orders — View all	Yes	Yes	No
Orders — Update Status	Yes	Yes	No

Customer — View list	Yes	Yes	No
Customer — Deactivate	Yes	No	No
Promotions — CRUD	Yes	Yes	No
Reports & Export	Yes	Yes	No
Staff — CRUD	Yes	No	No
Settings	Yes	Yes	No
Customer Menu & Cart	No	No	Yes
Customer Order History	No	No	Yes
Customer Profile	No	No	Yes

7. Feature Reference

7.1 Authentication

Sign In Window (gui/customer/customer_login_window.py)

The Sign In window is the single shared entry point for both portals. It first tries staff credentials; if that fails, it tries customer credentials. This allows the same dialog to serve all users without them selecting a role.

- **Staff login** — auth_service.login(email, password) looks up the staff table, verifies the bcrypt hash, updates last_login, and returns a StaffModel.
- **Customer login** — customer_auth_service.login(email, password) looks up the customers table and verifies the bcrypt hash, returning a CustomerModel.
- A **Create account** button registers a new customer directly from the Sign In window.
- Passwords are **never stored in plain text** — bcrypt with a random salt is used for every password.

7.2 Dashboard

Tab: Dashboard (gui/dashboard_tab.py)

The dashboard gives staff and admins a real-time snapshot of business performance.

KPI Cards (6 metrics):

Card	Value
Period Orders	Count of orders in the selected period

Total Orders	All-time order count
Period Revenue	Sum of total_amount in the selected period
Total Revenue	All-time revenue
Total Customers	Registered customer count
Avg. Order Value	Total revenue ÷ total orders

Period Selector: Today / This Month / This Year

Charts (requires PyQt6-Charts):

- **Bar chart** — Top products by quantity sold
- **Pie chart** — Revenue breakdown by category
- **Line chart** — Revenue over time in the selected period

Recent Orders table — The 10 most recent orders with order ID, customer, total, and status.

The dashboard auto-refreshes every 60 seconds and can be manually refreshed with the Refresh button.

7.3 Product & Category Management

Tabs: Products, Categories (gui/products_tab.py, gui/categories_tab.py)

Products

Action	Description
Add Product	Name, price, category, description, stock quantity, product image, and custom option groups
Edit Product	All fields; image can be replaced or cleared
Deactivate	Hides the product from the customer menu but keeps it in the database
Soft Delete	Marks is_deleted = True; product no longer appears in any list by default
Restore	Switches is_deleted back to False from the Deleted filter view

Filtering: All / Active / Inactive / Deleted, by category, and by name search.

Product Option Groups — Each product can have zero or more option groups (e.g., "Size" or "Temperature"). Each group has:

- A name
- Whether it is required
- Maximum choices (1 = radio buttons, >1 = checkboxes)

- Individual items with optional price additions

Categories

Action	Description
Add Category	Just a name; must be unique
Edit Category	Rename
Soft Delete	is_deleted = True; products in the category remain intact

7.4 Order Management (Staff)

Tab: Orders (gui/orders_tab.py)

New Order Window: gui/new_order_window.py

Placing a New Walk-In / Staff Order

6. Staff opens **New Order** from the Orders tab.
7. Products are displayed in a grid; clicking a card opens an **Options Dialog** if the product has option groups.
8. Items are added to an in-memory Cart object (services/cart_service.py).
9. Staff can enter an optional promo code; the code is validated in real time.
10. At checkout, staff selects: customer (search existing or walk-in), order type, and payment method.
11. On confirmation, order_service.place_order(...) is called — stock is checked, deducted, and the order is persisted.

Order Status Management

Staff update order status from the Orders tab. The allowed transitions are:

```
Pending         --> Processing, ReadyForPickup, ReadyForDelivery, Completed,
Cancelled
Processing      --> ReadyForPickup, ReadyForDelivery, Completed, Cancelled
ReadyForPickup --> Completed, Cancelled
ReadyForDelivery --> Completed, Cancelled
Completed       --> (terminal -- no further transitions)
Cancelled       --> (terminal -- no further transitions)
```

Every status change creates a row in order_status_history with the actor's name, the old status, the new status, and a timestamp.

Send Confirmation Email — Staff can trigger an order-confirmation HTML email to the customer's registered email address directly from the order detail view.

7.5 Customer Management

Tab: Customers (gui/customers_tab.py)

Staff and admins can search and view all registered customers. The list shows full name, email, contact number, total orders placed, and account status.

- **Deactivate** (Admin only) — Sets is_active = False; the customer can no longer log in.
- **Reactivate** (Admin only) — Restores access.
- **View Detail** — Shows the customer's full profile including delivery address and order history.

Walk-in customers (orders placed without a customer account) appear in the order list but not in the customer list.

7.6 Promotions & Discount Codes

Tab: Promotions (gui/promotions_tab.py)

Promotions provide discount codes that can be entered at checkout.

Promotion fields:

Field	Description
Name	Human-readable label (e.g., "Summer Sale")
Description	Optional detail shown to customers
Discount Type	Percentage (e.g., 10%) or FixedAmount (e.g., ₱50 off)
Discount Value	The numeric amount
Minimum Order Amount	Cart subtotal must be ≥ this value
Code	Unique redemption string (e.g., SAVE10)
Usage Limit	Maximum redemptions; leave blank for unlimited
Start / End Date	Date range; leave blank for no restriction
Active	Toggle to enable/disable without deleting
Product Scope	Optionally limit to specific products

Validation logic (services/promotion_service.py → validate_promotion_code):

12. Code exists in the database.
13. is_active is True.
14. Today is within start_date and end_date (if set).
15. used_count < usage_limit (if a limit is set).
16. Cart subtotal ≥ minimum_order_amount.
17. At least one item in the cart belongs to the product scope (if a scope is set).

If all checks pass, the discount is computed:

- **Percentage**: discount = subtotal × (value / 100)
- **FixedAmount**: discount = min(value, subtotal) (cannot go below zero)

7.7 Reports & Export

Tab: Reports (gui/reports_tab.py)

Reports have two sub-tabs: Products and Orders.

Products Report

Filters: category, status (All/Active/Inactive), optional date range.

Columns: ID, Name, Category, Price, Qty On Hand, Qty Sold, Revenue, Status.

Summary line: total products, total stock value, total revenue.

Orders Report

Filters: status, customer search, date range, order type.

Columns: Order ID, Customer, Email, Date, Items, Subtotal, Discount, Total, Type, Status, Payment, Email Sent.

Summary line: total orders, total revenue, total discounts given.

Exporting

Both reports can be exported to **CSV** or **PDF** directly from the toolbar. The export destination defaults to the exports/ directory and a file dialog confirms the path.

The export system uses an abstract base class design (see Section 10).

7.8 Staff Management

Tab: Staff (gui/staff_tab.py) — **Admin only**

Admins can add new staff members, edit existing ones, and deactivate accounts.

Fields: First Name, Last Name, Email, Role (Admin / Staff), Password (on create only), Active status.

Password validation on creation enforces:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit

7.9 Settings

Tab: Settings (gui/settings_tab.py)

Section	Fields
Store Information	Name, Contact, Email, Address
SMTP Email	Host, Port, Username, Password, From Name, Use TLS
Appearance	Theme (Light / Dark)

Settings are saved to config.json in the project root. A **Send Test Email** button verifies SMTP credentials before saving.

Theme changes apply immediately to the running application without a restart.

7.10 Customer Portal — Menu & Cart

Window: Customer Main Window (gui/customer/customer_main_window.py)

Menu Tab (menu_tab.py)

- **Hero banner** — Full-bleed image with a gradient overlay and welcome text.
- **Category filter** — Buttons to filter the product grid by category.
- **Product grid** — Responsive wrapping grid of product cards, each showing the image, name, price, and an "Add to Cart" button.
- Clicking **Add to Cart** opens an options dialog if the product has option groups; otherwise adds directly.

Cart Panel (cart_panel.py)

A persistent right-side panel showing the current cart. It is always visible in the customer portal.

Cart operations:

- Add items (with or without options)
- Adjust quantities with +/− buttons
- Remove individual items
- Clear the entire cart
- Apply a promo code (real-time validation)

Checkout flow (2-step dialog):

Step 1 — Order Details:

- Order type: Dine-In / Takeout / Delivery
- If Delivery: structured address form pre-filled from the customer's saved profile address.
- Payment method: Cash / GCash / Card (if Card: card number, expiry, CVV validated)

- Promo code field

Step 2 — Review & Confirm:

- Full order summary with subtotals, discount, and total
- Confirm button calls order_service.place_order(...), triggers the HTML confirmation email, and shows a success toast.

Cart badge — The shopping cart icon in the header shows a live count badge that updates as items are added/removed.

7.11 Customer Portal — Orders & Notifications

Orders Tab (gui/customer/orders_tab.py)

Shows the customer's own order history with order ID, date, items, total, order type, and current status. Customers can see the status history for each order.

Notifications Panel (gui/customer/notifications_panel.py)

A slide-in popup (bell icon in the header) showing a feed of recent order-status events pulled from order_status_history. Each entry shows:

- A colored icon (green for Completed, amber for Ready, red for Cancelled, etc.)
- A human-readable label (e.g., "Order is being prepared")
- The order ID
- A relative timestamp (e.g., "3 min ago", "2 days ago")

7.12 Customer Portal — Profile

Tab: Profile (gui/customer/profile_tab.py)

Customers can update their own account information:

Section	Fields
Account Information	First Name, Last Name (email is read-only)
Contact	Contact Number
Delivery Address	Street, City, Province, Postal Code, Landmark
Change Password	Current password (verified), New password, Confirm new password

Changes to the delivery address are automatically pre-filled in the next checkout's delivery form.

8. Input Validation Reference

All form fields are validated client-side before any database call. Invalid fields receive a red border via the error QSS property. Error messages are shown in a dialog.

Validation rules (services/validators.py):

Field Type	Rule
Required	Value must not be blank
Text Length	Configurable min/max; trims whitespace
Email	Must match ^[^@\s]+@[^@\s]+\.[^@\s]+$; max 254 characters
Phone	7–20 digits, optionally starting with +, with spaces or dashes
Name	Letters (including accented), apostrophes, hyphens, dots; max 60 characters
Postal Code	4–10 digits only
Password	Min 8 chars, ≥1 uppercase, ≥1 lowercase, ≥1 digit; max 128 chars
Promo Code	Uppercase letters, digits, _ and -; 3–20 characters

9. Email Notification System

Service: services/email_service.py

When an order is placed (by a customer or by staff for a customer with an email on file), the system sends an HTML order confirmation email via SMTP.

The email body includes:

- Store name in a branded header
- Customer first name greeting
- Order ID, date, type, and payment method
- Itemized table: product name, any selected options, quantity, unit price, subtotal
- Order totals: subtotal, discount, total
- A footer with the store name

SMTP configuration is stored in config.json and editable from the Settings tab. The send_order_confirmation() function catches all exceptions and returns (False, error_message) on failure so the order is never rolled back due to an email error.

10. Export System (OOP Design)

Module: services/exporters.py

The export system demonstrates classic object-oriented design through an abstract base class and concrete subclasses.

Abstract Base Class: BaseExporter

```
class BaseExporter(ABC):
    def __init__(self, data: list[dict], store_name: str): ...

    @abstractmethod
    def title(self) -> str: ...          # PDF heading

    @abstractmethod
    def headers(self) -> list[str]: ...  # Column headers for CSV and PDF

    @abstractmethod
    def format_row(self, row: dict) -> list: ...  # Convert dict to row values

    @abstractmethod
    def summary(self) -> dict: ...       # Aggregate metrics for PDF header

    def export_csv(self, filepath) -> None: ...  # Shared: writes CSV
    def export_pdf(self, filepath) -> None: ...  # Shared: builds PDF with
reportlab
```

export_csv and export_pdf are concrete methods that call the abstract hooks. This means subclasses only need to define what is unique (title, columns, row format, summary) and inherit the actual file-writing logic for free.

Concrete Subclasses

Class	Used For
ProductsExporter	Exports the products report
OrdersExporter	Exports the orders report

Both subclasses implement the four abstract methods and inherit export_csv and export_pdf without modification.

11. Configuration Reference

config.json is created automatically on first run. It supports the following keys:

```
{
  "store": {
    "name": "My Store",
    "contact": "+63-000-000-0000",
    "email": "store@example.com",
    "address": "123 Main St."
  },
  "smtp": {
    "host": "smtp.gmail.com",
    "port": 587,
    "username": "your@gmail.com",
    "password": "app-password",
    "use_tls": true,
    "from_name": "My Store"
  },
  "theme": "light"
}
```

theme can be "light" or "dark".

If config.json is deleted or becomes corrupted, the application falls back to the built-in defaults automatically.

12. Default Seed Data

On the very first launch, the database is populated with the following:

Staff Accounts

Role	Email	Password
Admin	admin@store.com	Admin@1234
Staff	maria@store.com	Staff@1234

Customer Accounts

Name	Email	Password
Juan Dela Cruz	juan@example.com	Customer@1234
Sofia Reyes	sofia@example.com	Customer@1234

Categories

Beverages · Snacks · Main Dishes

Sample Products

Product	Category	Price	Stock
Iced Coffee	Beverages	₱120.00	50
Lemonade	Beverages	₱90.00	40
Potato Chips	Snacks	₱55.00	100
Spaghetti	Main Dishes	₱180.00	25
Chicken Adobo	Main Dishes	₱220.00	20

Welcome Promotions

Code	Type	Value	Minimum Order
SAVE10	Percentage	10% off	None
MABUHAY50	Fixed Amount	₱50.00 off	₱500.00

13. Setup & Installation

Prerequisites

- Python 3.11 or newer
- Windows, macOS, or Linux with Qt 6 desktop support
- Internet access only if sending emails via SMTP

Step-by-step

Windows (cmd)

```
python -m venv .venv
.venv\Scripts\activate.bat
pip install -r requirements.txt
python main.py
```

Windows (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
python main.py
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python main.py
```

On first launch, inventory.db is created and seeded automatically. No separate database setup is needed.

Resetting to a Clean State

Delete inventory.db from the project root and relaunch. The database and all seed data will be recreated.

14. Sample Problems

The following scenarios walk through how the system handles realistic, day-to-day business situations.

Problem 1: Customer Orders Iced Coffee with a Size Option and Applies a Promo Code

Scenario: Sofia Reyes logs in to the Customer Portal, orders one large Iced Coffee, and applies the SAVE10 promo code.

Step-by-step:

18. Sofia opens the app and enters sofia@example.com / Customer@1234. The system routes her to the Customer Portal.
19. On the **Menu** tab, she sees the product grid. She clicks **Add to Cart** on the Iced Coffee card.
20. An **Options Dialog** appears showing the "Size" group (Small, Medium, Large). She selects **Large (+₱20.00)**. The unit price shown is ₱120.00 + ₱20.00 = ₱140.00.
21. She clicks **Add to Cart**. The cart badge in the header updates to 1.
22. In the Cart Panel, she types SAVE10 in the promo code field and clicks Apply.
23. The system calls validate_promotion_code("SAVE10", 140.00, [product_id]). All checks pass. Discount = 140.00 × 10% = ₱14.00.
24. The cart now shows: Subtotal ₱140.00 · Discount −₱14.00 · Total ₱126.00.
25. She clicks **Checkout**, selects Dine-In and Cash, and confirms.
26. order_service.place_order(...) runs: stock is checked, quantity_on_hand drops to 49, quantity_sold increases to 1, the order row is written with order_status = "Pending", and a status-history entry is recorded.
27. An HTML order-confirmation email is sent to sofia@example.com.

28. A green toast: "Order #X placed successfully!" appears for 2.4 seconds.

Problem 2: Staff Checks for Low-Stock Products and Restocks

Scenario: Maria (Staff) notices on the Dashboard that Spaghetti stock is running low. She needs to check all products and restock.

Step-by-step:

29. Maria logs in with maria@store.com / Staff@1234 and reaches the Staff Portal.
30. On the **Dashboard**, the "Period Revenue" and "Total Orders" KPI cards suggest Spaghetti is selling well.
31. She navigates to the **Products** tab, sets the filter to **Active**, and clicks **Run**.
32. She sorts by **Qty On Hand** ascending. She sees Spaghetti (25) and Chicken Adobo (20) at the top.
33. She double-clicks the **Spaghetti** row to open the Edit Product dialog.
34. She changes Quantity On Hand from 25 to 75 and clicks **Save**.
35. product_service.update_product(...) saves the updated quantity. The table refreshes.

Problem 3: Admin Creates a Limited-Time Promotion for the Holiday Weekend

Scenario: The Admin wants to run a holiday sale: 15% off all orders over ₱300, valid only December 24–26, maximum 100 uses, using the code HOLIDAY15.

Step-by-step:

36. Admin logs in and goes to the **Promotions** tab.
37. Clicks **Add Promotion**.
38. Fills in: Name: Holiday Weekend Sale; Description: 15% off all orders during the holiday weekend; Discount Type: Percentage; Discount Value: 15; Minimum Order Amount: 300; Code: HOLIDAY15; Usage Limit: 100; Start Date: 2026-12-24; End Date: 2026-12-26; Active: Yes.
39. Clicks **Save**. promotion_service.add_promotion(...) writes the row to the promotions table.
40. When a customer enters HOLIDAY15 on December 25 with a ₱450 cart, validate_promotion_code confirms all checks pass. Discount = ₱450 × 15% = ₱67.50.

Problem 4: Admin Generates a Monthly Revenue Report and Exports It to PDF

Scenario: At the end of the month, the Admin needs a PDF summary of all orders to share with the owner.

Step-by-step:

41. Admin opens the **Reports** tab and clicks the **Orders** sub-tab.
42. Sets filters: Status: All; Date From: 2026-05-01; Date To: 2026-05-31; Order Type: All.
43. Clicks **Run**. report_service.orders_report(...) queries all May orders.
44. The table populates. The summary line shows e.g., "Total Orders: 47 · Total Revenue: ₱18,340.00 · Total Discounts: ₱1,200.00".
45. Admin clicks **Export PDF**. A save dialog opens defaulting to the exports/ folder.
46. The system instantiates OrdersExporter(data, store_name) and calls .export_pdf(filepath).
47. A branded PDF report is saved to disk with a summary table at the top and the full orders table below.

Problem 5: Walk-In Customer Pays by GCash — Staff Places the Order

Scenario: A customer walks in without an account. A staff member takes the order for 2× Chicken Adobo and 1× Lemonade, paid via GCash.

Step-by-step:

48. Staff opens a **New Order** from the Orders tab.
49. Clicks the Chicken Adobo card → added directly. Adjusts quantity to 2. Line total: 2 × ₱220.00 = ₱440.00.
50. Clicks the Lemonade card → added directly. Line total: 1 × ₱90.00 = ₱90.00.
51. Cart shows: Subtotal ₱530.00.
52. Staff leaves the customer search blank (Walk-In, No Account).
53. Sets Order Type to **Dine-In**, Payment Method to **GCash**.
54. Confirms the order. order_service.place_order(customer_id=None, staff_id=maria_id, ...) runs. Stock checks pass. Quantities updated.
55. No email is sent (walk-in has no email). The system skips the email step gracefully.
56. The order appears in the Orders tab for Maria to track.

Problem 6: Admin Adds a New Product with Customization Options

Scenario: The store adds a new "Signature Milk Tea" (₱150.00) to the Beverages category, with a required Size option and an optional Add-On option.

Step-by-step:

57. Admin opens the **Products** tab and clicks **Add Product**.
58. Fills in: Name: Signature Milk Tea; Price: 150.00; Category: Beverages; Description: Premium milk tea with your choice of toppings; Quantity On Hand: 60.
59. In the Option Groups section, clicks **Add Group**: Group Name: Size; Required: Yes; Max Choices: 1; Items: Small (+₱0.00), Medium (+₱15.00), Large (+₱30.00).
60. Clicks **Add Group** again: Group Name: Toppings; Required: No; Max Choices: 3; Items: Tapioca Pearls (+₱15.00), Nata de Coco (+₱10.00), Grass Jelly (+₱10.00).

61. Clicks **Save**. product_service.add_product(...) writes the Product, then _replace_option_groups(...) writes the option group and item rows.
62. The product immediately appears in the product grid with radio buttons for Size and checkboxes for Toppings.

Problem 7: Staff Updates an Order Status and the Customer Sees It in Notifications

Scenario: After Problem 1, Maria needs to move Sofia's order from **Pending** to **Processing**, then to **Completed**.

Step-by-step:

63. Maria opens the **Orders** tab and finds Sofia's order (status: Pending).
64. She selects the order and clicks **Update Status**.
65. A dropdown shows allowed next statuses: Processing, ReadyForPickup, ReadyForDelivery, Completed, Cancelled.
66. She picks **Processing** and clicks Confirm.
67. order_service.update_order_status(order_id, "Processing", actor="Maria Santos") runs. Validates transition and updates status. Inserts a row in order_status_history.
68. Later, she updates to **Completed** — another history row is written.
69. Sofia logs back in and opens the **Notifications** bell icon. She sees two entries: "Order is being prepared · Order #X · 10 min ago" and "Order completed · Order #X · 2 min ago".

Problem 8: Admin Deactivates a Duplicate Customer Account

Scenario: The Admin finds two accounts for the same person. One is a duplicate with incorrect data. The Admin needs to deactivate the incorrect one.

Step-by-step:

70. Admin opens the **Customers** tab and searches by name.
71. Both accounts appear in the table. Admin identifies the duplicate by registration date and order count.
72. Admin selects the incorrect account and clicks **Deactivate** (visible only to Admins).
73. A confirmation dialog appears: "Deactivate this customer? They will no longer be able to log in."
74. Admin confirms. customer_service.deactivate_customer(customer_id) sets is_active = False.
75. The customer row shows status as Inactive.
76. If the deactivated customer tries to log in, customer_auth_service.login() checks is_active and returns None — they see an "Invalid credentials" message.
77. The Admin can **Reactivate** at any time if needed.